Lab 1 - Bazar.com

Distributed and Operating Systems
Noor Owaissi - 12217844 | Sara Abd Aldayem - 12340856

1. Program design

The project is a bookstore with four books. It has three separate services: Catalog, Order and Frontend. The client runs on the host computer. The services communicate using HTTP requests and return JSON. Frontend is the entry point for requests; it does not provide a graphical interface.

Catalog stores the book ID, title, topic, price and quantity. The first two books belong to distributed systems, and the other two belong to undergraduate school. The starting quantity is ten copies per book. The sample prices are 40, 50, 35 and 25. These starting values were chosen because the assignment does not give fixed prices or quantities.

Frontend has three operations. GET /search/<topic> asks Catalog for books in that topic. GET /info/<id> asks for the details of one book. POST /purchase/<id> sends a purchase request to Order. Search returns book IDs and titles. Information includes the price and available quantity.

Order handles buying one copy. It asks Catalog for the book details, then sends a stock update. If the update succeeds, Order saves a receipt in its own database and returns the order ID. A missing book returns 404. When there are no copies left, the purchase returns 409.

2. Data storage

Both backend services use SQLite. Catalog has a books table, and Order has an orders table. Orders include the book ID, title, price and purchase time. Each service owns its own database. Frontend does not read either database directly.

Catalog stores prices as integer cents, so a stored value of 5000 means 50.00. Database checks prevent negative prices and quantities. On startup, Catalog inserts missing books but does not replace existing rows. Restarting a service therefore keeps changes to prices and stock. Order initialization also keeps old orders.

Flask was used because it is small and supports the REST routes needed for this lab. SQLite was chosen instead of a larger database because the amount of data is small and the database is stored in one file. The client uses Python standard-library HTTP tools.

3. Updating stock and handling purchases

Catalog supports PATCH /update/<id>. The request can contain stock_delta, price, or both. A positive stock_delta adds copies and a negative value removes copies. For example, {"stock_delta": -1} removes one copy. Stock changes must be integers. Prices must be nonnegative numbers with no more than two decimal places. Invalid data returns 400.

A purchase may happen at the same time as another purchase. Checking the quantity and reducing it in separate transactions would allow both requests to see the last copy. Catalog avoids this by starting BEGIN IMMEDIATE before reading stock. It checks and updates the quantity inside that transaction. Other writers wait for it to finish. An update that would make stock negative is rejected.

Order first reads the book details and then requests the decrement. Catalog checks current stock again during the update, so an older lookup does not let Order remove stock that is no longer available. If a combined stock and price update is rejected, neither value changes.

4. Error handling and limitations

The services return JSON error messages. Frontend passes through backend status codes. Connection problems return 503, and an invalid backend JSON response returns 502. The client shows the error and stops. Purchase requests are not automatically retried because a timeout can happen after a change was already saved.

The stock decrement and the order insert happen in two different services. They are not one distributed transaction. If the order insert fails, Order tries to add the copy back to Catalog. If that request also fails, or if the service crashes before restoring stock, the data may need manual checking. This case is reported but is not fully solved.

The price saved by Order comes from its first lookup. If the price changes during the purchase, that saved price can be older than the new Catalog price. The system also does not have payment, shipping, authentication or cancellation features.

5. Possible improvements

A purchase ID could be used to recognize repeated requests and return the earlier result instead of buying twice. Catalog could reserve a copy with that ID and return the price attached to the reservation. A background task could check incomplete orders and either finish them or release the reserved copy. This would need extra stored states and recovery code.

Other improvements would be access control for stock updates, backups and clearer request IDs in logs. SQLite is suitable for this small lab, but a larger store would need to review database write limits and service capacity.

6. How to run

Install Docker Desktop and use Linux containers. Clone https://github.com/NOOROWAISSI/bazar.git and open a terminal in the bazar directory. Run docker compose up --build -d --wait. Run docker compose ps to check that all three services are healthy. Frontend is available at http://127.0.0.1:5000.

Compose runs three Linux containers. Waitress serves the Flask applications inside them. Catalog listens on port 5001, Order on 5002 and Frontend on 5000. Services use the names catalog and order to contact each other. Only Frontend is published to the host.

The host client needs Python 3.11 or newer. Create the environment using python -m venv .venv, then install requirements.txt with the environment interpreter. On Windows, run .venv/Scripts/python.exe client.py search "distributed systems". Use client.py info 2 to read book 2 and client.py purchase 2 to buy one copy. A successful purchase prints bought book followed by the title.

The README contains the exact PowerShell commands, API paths and an example of updating stock from inside the Catalog container. Use docker compose logs --no-color to view service messages. If port 5000 is occupied, set FRONTEND_PORT to another host port before starting Compose, and pass that address with the client --base-url option.

Catalog and Order use separate named volumes. docker compose down removes containers but keeps the databases. Starting Compose again uses the saved data. Adding -v to down deletes the volumes and should only be done when a full reset is intended. The normal desktop databases are separate from Docker data.

7. Testing

The combined test suite has 23 passing tests: three for Catalog, fourteen for Order and six for the client. It checks successful purchases, missing books, insufficient stock, input errors, saved orders, concurrent requests, connection failures and failed order storage. Run python -m unittest discover -s tests -v from the project environment.

The full Docker test ran all three real services and a client on the host. A purchase of book 2 reduced its quantity from ten to nine and saved an order. Thirty concurrent purchases for ten copies produced ten successes and twenty out-of-stock responses. Quantity stayed at zero rather than becoming negative.

The test removed and recreated the containers, then checked that stock and eleven order records were still saved. It also checked restocking, changing a price, another purchase and the error returned when Order is stopped. The tests used temporary data rather than the normal inventory. Actual client output and sample service logs are in docs/output.txt.